

Path2Prevention

A Lifestyle Recommendation System for Diabetes Prevention

(Comprehensive Presentation & Technical Report)

Slide 1: Project Introduction - Detailed Problem Statement

The Global Epidemic of Type 2 Diabetes Type 2 Diabetes (T2D) is currently one of the fastest-growing global health emergencies. According to the World Health Organization (WHO), hundreds of millions of people currently live with the condition, and millions die prematurely every year due to its complications (like cardiovascular disease, blindness, and kidney failure).

The "Invisible" Pre-Diabetes Phase Before developing full T2D, individuals enter a stage called "pre-diabetes" where blood sugar levels are higher than normal but not high enough for a clinical diagnosis. This phase is almost entirely asymptomatic. Because there are no visible symptoms, millions of people live with high-risk lifestyle habits completely unaware of the damage occurring to their metabolic systems.

The Demographic Vulnerability (South Asian Populations) The problem is significantly worse for specific demographics. For example, South Asian populations are known to have a higher genetic predisposition to insulin resistance. They often develop T2D at much younger ages and at a lower Body Mass Index (BMI). While Western medical guidelines consider a BMI of 25 to be "overweight", South Asian populations often show severe metabolic risk at a lower BMI of 23. Furthermore, they suffer from higher rates of central adiposity (dangerous belly fat) combined with lower muscle mass.

Currently, generic health apps ignore these crucial ethnic and medical demographic differences, providing unsafe or inaccurate health advice.

Slide 2: Project Introduction - Deep Need of the Project

The Failure of Generic Advice When individuals visit a doctor, they are often told universally generic advice: "Eat less, move more." This advice is notoriously ineffective because it lacks personalization. It does not account for an individual's specific daily habits, cultural dietary patterns, sleep quality, or stress levels.

The Healthcare Burden Our modern healthcare system is inherently "reactive." Doctors treat diseases after symptoms appear. By the time a clinical diagnosis of Type 2 Diabetes is made, up to 50% of the insulin-producing beta cells in the pancreas may already be destroyed. There is a desperate need for a "proactive" system that prevents the disease before it happens.

Our Innovative Solution Path2Prevention fills this massive gap by serving as a localized, proactive digital health assistant. There is a critical need for an automated, software-based Rule Engine that analyzes small, daily behavioral signals (e.g., glasses of water, minutes of walking, hours of sleep) and calculates a real-time Risk Score. By automatically flagging high-risk habits and prioritizing exactly which changes the user should make, Path2Prevention effectively digitizes preventive medicine and makes it accessible for free on any mobile device.

Slide 3: Project Introduction - Core Objectives

Primary Objectives:

1. **Develop an Intelligent Rule Engine:** Build a backend computation system (`pipeline.js`) that translates complex medical guidelines from the World Health Organization (WHO), International Diabetes Federation

(IDF), and American Diabetes Association (ADA) into logical software rules.

2. **Dynamic Risk Scoring:** Provide users with a highly accurate, daily Risk Score (scaled from 0 to 100) that mathematically adjusts based on family history, localized BMI thresholds, waist circumference, and daily behavioral deficits.
3. **Targeted Interventions:** Replace overwhelming medical advice with exactly 3 to 5 highly-prioritized, actionable daily recommendations tailored specifically to the user's weakest areas derived from their logs.

Secondary Objectives:

1. **Visual Analytics:** Drive continuous user engagement by providing historical line charts, pattern correlations, and interactive visual data representations.
 2. **"What-If" Predictive Simulator:** Build an interactive risk simulator tool allowing users to visually see how improving their sleep or walking more will mathematically reduce their diabetes risk in the future.
 3. **"Clinical Serenity" Design:** Implement a calming, medical-grade UI/UX design (utilizing Glassmorphism and specific color psychology) that reassures users rather than causing anxiety.
-

Slide 4: Planning of Project Work - Project Timeline (Gantt Chart)

(Represent this strictly as a visual Gantt chart in the Presentation)

- **Phase 1: Research & Blueprinting (Week 1)**
 - Requirement analysis and demographic medical research (IDF guidelines).
 - System Architecture mapping and MongoDB database schema design.
 - **Phase 2: Core Backend Engine Development (Week 2)**
 - Constructing the Express.js REST API.
 - Developing the `pipeline.js` algorithm (calculating sub-scores for activity, sleep, diet, and physical measurements).
 - **Phase 3: Frontend Interface & UI Generation (Week 3)**
 - Integrating Stitch-inspired Material Design UI.
 - Connecting HTML5/Vanilla JS frontend with backend JWT-secured APIs (Login/Register, Daily Logs, Profile).
 - **Phase 4: Analytics, Simulators, & Exports (Week 4)**
 - Building Chart.js historical dashboard graphs.
 - Developing the interactive What-If Score Simulator.
 - Building the `export.js` engine for generating downloadable HTML/PDF offline reports.
 - **Phase 5: System Audit, Testing & Deployment (Week 5)**
 - End-to-End User Flow Quality Assurance testing.
 - Mathematical validation of the 128-point custom denominator in the rule scoring system.
 - Dead code removal, edge-case fixing, and comprehensive presentation documentation.
-

Slide 5: Team Structure & Roles

(Customize individual names formatting as needed)

- **Project Manager / Team Leader:**
 - **Responsibilities:** Oversees the overall system architecture, product vision, and ensures that the software algorithms perfectly align with clinical diagnostic guidelines (WHO/IDF). Manages the timeline and agile sprint delivery.
- **Full-Stack Developer:**

- **Responsibilities:** Writes all backend logic including Mongoose data schemas, Express.js routing, and the heavy-lifting custom Rule-Base Engine. Also responsible for writing Vanilla JS asynchronous frontend network requests (`api.js`) and state management.
 - **UI/UX Designer:**
 - **Responsibilities:** Crafts the "Clinical Serenity" visual aesthetic. Selects custom CSS properties, ensuring the UI uses modern Glassmorphism, highly-legible typography, color-coded health meters, and responsive layouts for mobile WebView compatibility.
 - **Quality Assurance (QA) Tester:**
 - **Responsibilities:** Conducts rigid validation testing. Mathematically calculates and validates the risk scoring outputs (ensuring South Asian Action Point thresholds successfully impact the pipeline). Performs deep system audits to eliminate routing errors and UI crashes before production deployment.
-

Slide 6: Design Methodology - Development Model

Model Used: Agile Methodology

Why we selected Agile over Waterfall:

1. **High Iteration Speed:** Building a complex mathematical health rule-engine requires constant mathematical tweaks. Waterfall requires perfect upfront planning, but Agile allowed the team to build feature functionality in modular, 1-week sprints (e.g., building the Log Form, then the Dashboard, then the Insights Analytics).
 2. **Continuous Feedback Loop:** During development, we realized generic Western BMI thresholds were inaccurate for our target demographics. Agile's flexibility allowed us to dynamically pivot our entire database pipeline to accommodate localized South Asian IDF thresholds dynamically without delaying the project.
 3. **Modular Component Architecture:** Agile allowed us to develop the frontend features (Dashboard, Insights, Settings) and test them entirely independently as their specific backend APIs were finished.
-

Slide 7: Design Methodology - Tools Used

- **Source Code & Version Control:** Git, GitHub (for branch handling and commit histories).
 - **API Testing & Debugging:** Postman (for testing raw JSON payload responses) and Browser Chrome Developer Tools (Network/Console monitoring).
 - **UI/UX Prototyping & AI Assistance:** Figma (for early UI Mockups), and Stitch MCP (AI Interface Assistance mapping advanced styling).
 - **Integrated Development Environment (IDE):** Visual Studio Code (VS Code) utilizing Node extensions and Prettier formatting.
 - **Documentation:** Markdown (MD) for dynamic documentation and Marp CLI for programmatic presentation generation.
-

Slide 8: Design Methodology - Technologies Used

1. Frontend (Client-Side Architecture):

- **HTML5 / CSS3:** Utilizes extensive custom CSS variables (Root tokens), Flexbox/Grid for layout, and advanced Glassmorphism design aesthetics (blurred backdrops and gentle shadows).
- **Vanilla JavaScript (ES6+):** Pure DOM manipulation and asynchronous network fetching (`async/await`) completely independent of heavy frameworks like React or Angular to ensure maximum performance and lightweight serving.

- **Chart.js / HTML Canvas:** For rendering highly-responsive, multi-dataset chronological line charts and visual analytic patterns.

2. Backend (Server-Side Architecture):

- **Node.js & Express.js:** Fast, non-blocking, event-driven HTTP framework acting as the core traffic router for RESTful endpoints.

3. Database Layer:

- **MongoDB (NoSQL Document Store):** Selected due to the flexible, unstructured nature of evolving medical data.
- **Mongoose ODM:** Provides crucial schema definitions, automated timestamps, and complex aggregation capabilities for timeline queries.

4. Additional Ecosystem Tools:

- **JSON Web Tokens (JWT) & Bcrypt:** For highly-secured hash authentication and user identity middleware verification.

Slide 9: System Architecture - Block Diagram

(Visual Representation of the Data Flow between Components)

1. User Interaction (PWA Browser/Mobile): The user accesses the application and inputs daily habits via secure HTTPS. ↓ **2. Frontend Interface (HTML, CSS, JS, Chart.js):** The UI layer captures the data and routes it securely to the system logic utilizing JSON Web Tokens (JWT). ↓ **3. Backend Processing (Node.js + Express.js API):** The core traffic router authenticates the request, processes the payload, and delegates it to the algorithmic `pipeline.js` component. ↓ (Read/Write) **4. Database (MongoDB Collections):** The Mongoose ODM inserts the new User Logs and retrieves aggregated historical patterns required for deep calculation. ↓ **5. System Output:** The analytical Engine produces the final Output: Dynamic Risk Scores, visual intervention alerts, and exportable PDF/CSV reporting.

Slide 10: Conceptual Demonstration - How the System Actually Works

Step 1: Onboarding Baseline A user creates a highly secure account using JWT authentication. They complete a 4-step onboarding assessment gathering baseline clinical markers: height, weight, biological sex, and crucial family history of diabetes—which establishes their baseline genetic multiplier.

Step 2: Micro-Habit Logging Instead of counting every single calorie (which causes user burnout), the system relies purely on "Micro-Habits". The user utilizes minimalist sliders and toggles to submit a daily log covering steps, sleep duration, sugary drink intake, fast food consumption, and subjective stress levels.

Step 3: Algorithmic Pipeline Calculation Upon submission, the Node.js backend immediately triggers the `pipeline.js` computation engine. The engine performs high-level mathematical aggregations on the last 7 to 14 days of logs.

- *Example:* The engine sees the user logged 0 minutes of moderate activity. It mathematically scores this 0 against the WHO medical target of 150 minutes per week, calculating a severe "Activity Deficit Penalty" out of 20 points.
- *Example:* The engine notes a South Asian male has a waist circumference of 92cm. It checks the IDF threshold (<90cm) and automatically assigns a risk penalty.

Step 4: Priority Intervention Selection The engine calculates the final global Risk Score (e.g., 65/100 -> "High Risk") and then queries the extensive medical Rule Base database. Rather than overwhelming the user, it extracts ONLY the

top 3-5 highest-priority lifestyle interventions specifically linked to the penalties generated in Step 3 (e.g., "Cut Sugary Drinks - Risk lowered by stabilizing liver enzymes").

Step 5: Visual Dashboard Return The frontend dashboard instantly re-renders. The user views their color-coded Risk Gauge update in real-time. They can switch to the "Insights" tab to view interactive Chart.js graphs tracking their step-count versus sleep-quality correlations dynamically over a 30-day period.

Slide 11: Conceptual Demonstration - Flowchart of the Process

Step 1. START: User Registration / Login The user securely enters the application.

Step 2. CONDITION: Is Onboarding Complete?

- *If No:* Route to Onboarding Diagnostic Survey to save the initial Profile.
- *If Yes:* Route directly to the main Dashboard Landing UI.

Step 3. ACTION: Daily Submission User submits their Daily Habit Log Form tracking their micro-habits.

Step 4. PROCESS: Backend Validates Data The server sanitizes the input and enforces a log lock to ensure data immutability.

Step 5. TRIGGER: Execution of Rule Engine The system triggers the `pipeline.js` algorithm to calculate global metabolic deficits.

Step 6. DECISION: Medical Safety Check

- *If Risk Score exceeds Safety Limit (e.g., extreme HbA1c):* Issue an Urgent Medical Alert Override.
- *If Safe:* Evaluate the Standard Medical Habit Target Rules.

Step 7. DATABASE: Store New Outputs Generate the new Global Risk Score & personalized priority interventions in MongoDB.

Step 8. END: Render Updates The user interface re-renders the Dashboard Insights, Real-Time Recommendations, and Analytics Charts.